

TEAM D — Market Data Service · Next.js Frontend

Owner: Arham Apon Utsho (220042153) Theme: everything the user sees — live prices, candlestick charts, and the whole trading UI.

Give this whole file to Claude Code in the repo root and say: "Read TEAM_D_MARKETDATA_AND_FRONTEND.md and implement the Market Data Service and the frontend completely. *Libs/common/* already exists — read it, import from it, do not rewrite it."

You depend on nobody. Market Data only consumes `trade.executed` (fake it with the script in §6). The frontend is built against the frozen gateway contract in §1.4 and runs against the mock API in §7 until the real backend lands.

1. THE SHARED CONTRACT (identical in all four team documents — never change unilaterally)

1.1 Your ports and databases

Component	Container	Port	Database
Market Data Service	market-data-service	8005	writes postgres-market-primary:5432/market_db (host 5436)
			reads postgres-market-replica:5432/market_db (host 5437)
Frontend	frontend	3000	—

Others: gateway 8000, user 8001, account 8002, order 8003, matching-engine 8004, portfolio 8006, notification 8007, redis 6379, rabbitmq 5672/15672.

The master–slave split is a graded requirement of the proposal. Team A makes streaming replication work; **you** must actually route reads to the replica and writes to the primary.

1.2 Money & quantity rules

- Money is `Decimal`, 4 dp, `ROUND_HALF_UP`. **Never float** on the Python side. Postgres `NUMERIC(18,4)`.
- Money crosses the wire as a **JSON string**: `"195.5000"` — with **one exception**, below.
- **Chart candles are the exception**: `lightweight-charts` needs JS numbers, so `/market/candles` returns `open/high/low/close` as **numbers** and `time` as an **epoch-seconds integer**. Everything else (quotes, prices in tables) stays a string. Say this explicitly in your `SERVICE_NOTES.md` — it is a deliberate, documented deviation, not sloppiness.
- Quantity is a positive **int**. Postgres `BIGINT`.
- Symbols: `common.symbols.SYMBOLS`, `SEED_PRICES`, `normalize_symbol`.

1.3 Auth

JWT HS256 via `common.security`. **Every Market Data read endpoint is public** (browsing prices needs no login — that's why the gateway whitelists `/api/market/`). `/internal/*` uses `require_internal_key`.

1.4 Gateway routing table — this is the frontend's entire world

Base URL: `NEXT_PUBLIC_API_URL = http://localhost:8000`. WS base: `NEXT_PUBLIC_WS_URL = ws://localhost:8000`.

Method	Path	Auth	Body / Response
POST	/api/auth/register	—	{email,username,password,full_name?} → {user, access_token, token_type, expires_in}
POST	/api/auth/login	—	form-encoded username (email or username) + password → {access_token, token_type, expires_in, user}
POST	/api/auth/login-json	—	{email_or_username, password} → same
GET	/api/users/me	JWT	user
GET	/api/account/balance	JWT	{user_id, cash_balance, held_balance, available_balance, currency} (strings)
POST	/api/account/deposit	JWT	{amount:"10000.00"} → balance
GET	/api/account/transactions?limit&offset	JWT	list
POST	/api/orders	JWT	{symbol, side:"BUY" "SELL", order_type:"LIMIT" "MARKET", price?:"195.50", quantity:10, client_order_id?} → 201 order
GET	/api/orders?status&symbol&limit&offset	JWT	list
GET	/api/orders/{id}	JWT	order
DELETE	/api/orders/{id}	JWT	202 {status:"CANCEL_PENDING"}
GET	/api/book/{symbol}?depth=10	—	{symbol,bids:[{price,quantity,orders}],asks:[...],best_bid,best_ask,spread,ts}
GET	/api/market/symbols	—	list of {symbol,name,last_price,change,change_pct}
GET	/api/market/quote/{symbol}	—	{symbol,price,change,change_pct,volume_24h,ts}
GET	/api/market/candles/{symbol}?interval=1m 5m&limit=200	—	[{time,open,high,low,close,volume}] — numbers , ascending
GET	/api/market/trades/{symbol}?limit=50	—	recent trades
GET	/api/portfolio	JWT	holdings + totals
GET	/api/portfolio/pnl	JWT	P&L summary
GET	/api/notifications?unread_only&limit&offset	JWT	list
GET	/api/notifications/unread-count	JWT	{count}
POST	/api/notifications/{id}/read	JWT	ok
POST	/api/notifications/read-all	JWT	{marked}
WS	/ws/market?symbols=AAPL,TSLA	—	live ticks

Order object (what the frontend renders):

```
{
  "id", "user_id", "symbol", "side", "order_type", "price", "quantity", "filled_quantity",
  "avg_fill_price", "status", "reject_reason", "created_at", "updated_at"
}
```

Statuses: `PENDING`, `NEW`, `PARTIALLY_FILLED`, `FILLED`, `CANCEL_PENDING`, `CANCELLED`, `REJECTED`.

Errors are always `{"detail": "human readable message"}` with a real HTTP status (400 validation, 401 auth, 403 not yours, 404 missing, 409 business conflict, 429 rate limited, 503 upstream down). **Show detail to the user verbatim** — it is written to be user-facing.

1.5 Event contract — you consume exactly one event

Exchange `exchange.events` (topic, durable). Envelope:

```
{
  "event_id": "<uuid4>",
  "event_type": "trade.executed",
  "occurred_at": "...Z",
  "version": 1,
  "payload": {...}
}
```

Your queue: `q.marketdata.trade_executed` bound to `trade.executed`.

```
{
  "trade_id": "uuid",
  "symbol": "AAPL",
  "price": "195.5000",
  "quantity": 5,
  "buy_order_id": "uuid",
  "sell_order_id": "uuid",
  "buyer_user_id": "uuid",
  "seller_user_id": "uuid",
  "aggressor_side": "BUY",
  "buy_order_remaining": 0,
  "sell_order_remaining": 3,
  "buy_order_limit_price": "196.0000",
  "executed_at": "2026-08-08T12:00:00.000000Z"
}
```

1.6 Redis keys — you own the writes to these

<code>md:last_price:{SYMBOL}</code>	<code>"195.5000"</code>	<code><- Portfolio reads it for unrealized P&L</code>
<code>md:quote:{SYMBOL}</code>	<code>{"price", "quantity", "ts"}</code>	<code><- your own quote cache</code>
<code>CHANNEL md:ticks</code>	<code>{"symbol", "price", "quantity", "ts"}</code>	<code><- pub/sub fan-out to WS clients</code>

If you stop writing `md:last_price`, Portfolio's unrealized P&L silently goes to zero and Order Service rejects every MARKET order. This is your contract with two other services.

1.7 Every service exposes

`GET /health` (no DB), `GET /ready` (primary + replica + redis + broker), Swagger `/docs, version="1.0.0"`.

1.8 Dockerfile template — Python service (build context is the REPO ROOT)

```
FROM python:3.11-slim
WORKDIR /app
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1 PYTHONPATH=/app:/app/libs
COPY requirements-base.txt ./
COPY services/market-data-service/requirements.txt ./service-requirements.txt
RUN pip install --no-cache-dir -r service-requirements.txt
COPY libs /app/libs
COPY services/market-data-service/app /app/app
COPY services/market-data-service/alembic.ini /app/alembic.ini
```

```
COPY services/market-data-service/alembic /app/alembic
COPY services/market-data-service/entrypoint.sh /app/entrypoint.sh
RUN chmod +x /app/entrypoint.sh
EXPOSE 8005
CMD ["/app/entrypoint.sh"]
```

`entrypoint.sh` — **LF endings only**, and migrations run against the **primary**:

```
#!/bin/sh
set -e
alembic upgrade head
exec uvicorn app.main:app --host 0.0.0.0 --port "${SERVICE_PORT}"
```

2. MARKET DATA SERVICE (port 8005)

2.1 Two engines — the read/write split

```
from common.config import settings
from common.db import make_engine, make_sessionmaker

write_engine = make_engine(settings.DATABASE_URL) # primary
replica_url = settings.DATABASE_REPLICA_URL or settings.DATABASE_URL # graceful fallback
read_engine = make_engine(replica_url)
WriteSession, ReadSession = make_sessionmaker(write_engine), make_sessionmaker(read_engine)
```

- **Every write** (trade insert, candle upsert, symbol seed, Alembic) → `WriteSession` / `DATABASE_URL`.
- **Every chart/history read** (`/market/candles`, `/market/trades`, `/market/symbols`) → `ReadSession`.
- At startup, log which URL each engine got, and log `SELECT pg_is_in_recovery()` from the read engine — `true` proves you are really on the replica. **Screenshot this for the report.**
- If the replica is unreachable at request time, catch `OperationalError`, log a warning, and retry once on `WriteSession`. Never 500 a chart because a replica is lagging.

⚠ **Writing to the replica raises cannot execute INSERT in a read-only transaction.** If you see that error, a write path is using the read session.

2.2 Models

```
class Symbol(Base):
    __tablename__ = "symbols"
    symbol = String(10) pk; name = String(120); seed_price = Numeric(18,4); created_at

class Trade(Base):
    __tablename__ = "trades"
    id = uuid pk
    trade_id = UUID unique not null # idempotency lives here
    symbol = String(10) not null
    price = Numeric(18,4) not null
    quantity = BigInteger not null
    aggressor_side = String(4)
    executed_at = DateTime(timezone=True) not null
    __table_args__ = (Index("ix_trades_symbol_time", "symbol", "executed_at"),)
```

```
class Candle(Base):
    __tablename__ = "candles"
    id = uuid pk
    symbol = String(10) not null
    interval = String(4) not null # '1m' | '5m'
    bucket_start = DateTime(timezone=True) not null
    open = high = low = close = Numeric(18,4) not null
    volume = BigInteger default 0; trade_count = Integer default 0
    __table_args__ = (
        UniqueConstraint("symbol", "interval", "bucket_start", name="uq_candle_bucket"),
        Index("ix_candles_lookup", "symbol", "interval", "bucket_start"),
    )
```

Seed the 8 `symbols` rows from `common.symbols` on startup if the table is empty (idempotent — `ON CONFLICT DO NOTHING`).

2.3 The `trade.executed` handler (write path — primary only)

```
async def handle_trade(env):
    if await already_processed(redis, "marketdata", env["event_id"]): return
    try:
        p = env["payload"]
        sym = normalize_symbol(p["symbol"])
        price = to_money(p["price"]); qty = int(p["quantity"])
        ts = datetime.fromisoformat(p["executed_at"].replace("Z", "+00:00"))
        async with WriteSession() as s, s.begin():
            await s.execute(insert(Trade).values(...).on_conflict_do_nothing(index_elements=
["trade_id"]))
            for interval, minutes in (("1m", 1), ("5m", 5)):
                await upsert_candle(s, sym, interval, floor_bucket(ts, minutes), price, qty)
            await redis.set(f"md:last_price:{sym}", money_str(price))
            await redis.set(f"md:quote:{sym}", json.dumps({"price": money_str(price), "quantity":
qty,
                                                    "ts": ts.isoformat()}))
            await redis.publish("md:ticks", json.dumps({"symbol": sym, "price": money_str(price),
                                                    "quantity": qty, "ts":
int(ts.timestamp())}))
        except Exception:
            await clear_processed(redis, "marketdata", env["event_id"]) # let the retry re-run
            raise
```

`floor_bucket(ts, minutes)` — always UTC:

```
ts = ts.astimezone(timezone.utc).replace(second=0, microsecond=0)
return ts.replace(minute=(ts.minute // minutes) * minutes)
```

`upsert_candle` — one atomic statement, no read-modify-write race:

```
from sqlalchemy.dialects.postgresql import insert as pg_insert
stmt = pg_insert(Candle).values(id=uuid4(), symbol=sym, interval=interval,
bucket_start=bucket,
                                open=price, high=price, low=price, close=price,
```

```

                                volume=qty, trade_count=1)
await s.execute(stmt.on_conflict_do_update(
    index_elements=["symbol", "interval", "bucket_start"],
    set_={"high": func.greatest(Candle.high, stmt.excluded.high),
        "low": func.least(Candle.low, stmt.excluded.low),
        "close": stmt.excluded.close,
        "volume": Candle.volume + stmt.excluded.volume,
        "trade_count": Candle.trade_count + 1}))

```

open is only ever set on INSERT — that is exactly what `on_conflict_do_update` gives you, since `open` is absent from `set_`. **Build 5m candles from trades directly, never by re-aggregating 1m candles.** Deriving from 1m double-counts on any replay and gets the open wrong.

2.4 REST endpoints (all reads from the **replica**)

Method	Path	Notes
GET	<code>/market/symbols</code>	8 rows: {symbol,name,last_price,change,change_pct,volume_24h}. last_price from Redis via one MGET; change vs the close 24 h ago (one grouped query)
GET	<code>/market/quote/{symbol}</code>	Redis <code>md:quote</code> first; on a miss, the newest trade from the replica; on a second miss, <code>SEED_PRICES</code> with "stale": true
GET	<code>/market/candles/{symbol}?interval=1m 5m&limit=200&from=&to=</code>	chart data. limit ≤ 1000. Ascending by time. time = epoch seconds int, OHLC as numbers
GET	<code>/market/trades/{symbol}?limit=50</code>	newest first, prices as strings
GET	<code>/market/stats</code>	one-shot market summary for the dashboard
GET	<code>/internal/replication-status</code>	{"is_replica": bool, "replica_url_configured": bool, "lag_bytes": int null} — your proof for the report
WS	<code>/ws/market?symbols=AAPL,TSLA</code>	live ticks

Gap filling on candle reads. A minute with no trades has no row, and a chart with holes looks broken. After fetching, forward-fill: for each missing bucket between the first and last, emit {open=high=low=close=previous close, volume:0}. Cap the fill at limit buckets so a 2-day gap can't generate a million objects.

2.5 WebSocket `/ws/market`

```

@app.websocket("/ws/market")
async def ws_market(ws: WebSocket):
    await ws.accept()
    symbols = {normalize_symbol(s) for s in ws.query_params.get("symbols","").split(",") if
s.strip()}
    if not symbols: symbols = set(SYMBOLS) # no filter => everything
    conn = Connection(ws, symbols); MANAGER.add(conn)
    try:
        for sym in symbols: # immediate snapshot so the UI is never
blank
            price = await redis.get(f"md:last_price:{sym}")
            if price: await
ws.send_json({"type":"tick","symbol":sym,"price":price,"ts":now()})
            while True:

```

```

        msg = await ws.receive_json() #
        {"action": "subscribe" | "unsubscribe", "symbols": [...]}
        ...update conn.symbols...
    except WebSocketDisconnect:
        pass
    finally:
        MANAGER.remove(conn)

```

A **single** background task per process subscribes to the Redis `md:ticks` channel and fans out to matching local connections:

```

async def tick_pump():
    pubsub = redis.pubsub()
    await pubsub.subscribe("md:ticks")
    async for message in pubsub.listen():
        if message["type"] != "message": continue
        tick = json.loads(message["data"])
        await MANAGER.broadcast(tick["symbol"], {"type": "tick", **tick})

```

Start it in the lifespan with `asyncio.create_task`; cancel it on shutdown. **This Redis Pub/Sub fan-out is exactly the pattern the proposal asks for** — it is what lets you run multiple Market Data replicas behind the gateway.

`MANAGER.broadcast` must be crash-proof: iterate over a **copy** of the connection set, wrap each `send_json` in `try/except`, and drop any connection that raises. One dead browser tab must never stop the fan-out.

Also send a heartbeat `{"type": "ping", "ts": ...}` every 20 s so proxies don't kill idle sockets.

2.6 Edge cases — Market Data Service

1. **Writing to the replica** → `cannot execute INSERT in a read-only transaction`. Every write goes through `WriteSession`. Assert it once at startup by attempting a write on the read engine inside a `try` and logging the (expected) failure — proof for the report.
2. **Replica lag**. A candle written 200 ms ago may not be readable yet. The frontend also applies the live WS tick, so the user sees no lag. Document it as intended CQRS behaviour.
3. **Replica down** → catch `OperationalError`, log, retry once on the primary. Never 500 a chart.
4. **DATABASE_REPLICA_URL unset** → falls back to the primary and everything still works. This is your insurance if Team A's replication is not ready by hour 18.
5. **Duplicate trade events** → `trade_id` unique + `ON CONFLICT DO NOTHING` + Redis idem. Without this a redelivery double-counts volume.
6. **Concurrent candle upserts** for the same bucket → `ON CONFLICT DO UPDATE` in one statement. A read-modify-write here loses trades under load.
7. **open overwritten on update** → the single most common candle bug. `open` must be absent from `set_`.
8. **Timezones**. Everything is UTC. `datetime.fromisoformat` on `"...Z"` fails on Python < 3.11 — use `.replace("Z", "+00:00")`. Naive datetimes in `bucket_start` will silently shift the whole chart.
9. **time must be epoch seconds**. `lightweight-charts` silently renders nothing for ISO strings or millisecond timestamps.
10. **Duplicate or unsorted time values** make `lightweight-charts` throw `Assertion failed: data must be asc ordered by time`. Sort ascending and de-duplicate before returning.
11. **Empty candle set** (a brand-new symbol) → return `[]`, and the UI shows "No data yet" rather than a broken chart.
12. **Unknown symbol** → 404 `{"detail": "unknown symbol"}`. Validate with `normalize_symbol` before any query.
13. **limit abuse** (`?limit=999999`) → clamp to 1000.
14. **WS client disconnects mid-broadcast** → per-connection `try/except`, drop, continue.
15. **Slow WS client** → don't buffer forever; if `send_json` raises or times out, drop the connection.
16. **WS with no symbols param** → subscribe to all 8, don't crash on the empty string.

17. **Redis restarts** → `md:last_price` is gone. On startup, warm the cache from the newest trade per symbol on the replica; if there are no trades, use `SEED_PRICES`. **Portfolio and Order Service both depend on this key existing.**
18. **`pubsub.listen()` dies** when Redis restarts → wrap `tick_pump` in a `while True` with `reconnect + 2 s` backoff.
19. **Float in the tick payload** → keep prices as strings on the wire; the frontend does `Number()` for display only.
20. **24h change with no history** → `change: "0.0000", change_pct: 0.0`, never `null`, never a divide-by-zero.

3. FRONTEND — Next.js (port 3000)

3.1 Stack (pin these)

```
next@14.2.18  react@18.3.1  typescript@5.6.3
tailwindcss@3.4.15  lightweight-charts@4.2.0
```

⚠️ **Pin `lightweight-charts` to 4.x.** v5 removed `chart.addCandlestickSeries()` in favour of `chart.addSeries(CandlestickSeries, ...)`; every tutorial and every LLM's memory is v4. Mixing them produces `addCandlestickSeries is not a function`.

App Router. No state library — `AuthContext` + `useState/useEffect` is enough.

3.2 Structure

```
frontend/
  app/
    layout.tsx          AuthProvider, Navbar, toast host
    page.tsx             dashboard
    login/page.tsx  register/page.tsx
    market/[symbol]/page.tsx  chart + book + trades + order ticket
    portfolio/page.tsx
    orders/page.tsx
    wallet/page.tsx
  components/
    Navbar.tsx  NotificationBell.tsx  PriceTicker.tsx
    CandleChart.tsx      'use client', dynamic(ssr:false)
    OrderBook.tsx  RecentTrades.tsx  OrderTicket.tsx
    HoldingsTable.tsx  OrdersTable.tsx  Toast.tsx  Protected.tsx
  lib/
    api.ts      typed fetch wrapper
    auth.tsx    AuthContext (token in localStorage)
    ws.ts       reconnecting WebSocket client
    format.ts   money/qty/percent formatting
    types.ts    mirrors $1.4 exactly
  Dockerfile  next.config.js  tailwind.config.ts  package.json  .env.local.example
```

3.3 `lib/api.ts` — one wrapper, every call goes through it

```
const BASE = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8000";

export async function api<T>(path: string, init: RequestInit = {}): Promise<T> {
  const token = typeof window !== "undefined" ? localStorage.getItem("mse_token") : null;
  const res = await fetch(`${BASE}${path}`, {
    ...init,
    headers: {
```

```

    "Content-Type": "application/json",
    ...(token ? { Authorization: `Bearer ${token}` } : {}),
    ...init.headers,
  },
  cache: "no-store",
});
if (res.status === 401) {
  localStorage.removeItem("mse_token");
  if (!location.pathname.startsWith("/login")) location.href = "/login";
  throw new ApiError(401, "Session expired, please log in again");
}
if (!res.ok) {
  const body = await res.json().catch(() => ({}));
  throw new ApiError(res.status, body.detail ?? `Request failed (${res.status})`);
}
return res.status === 204 ? (undefined as T) : res.json();
}

```

Login is the one exception — it is `application/x-www-form-urlencoded` (OAuth2 password flow). Use `/api/auth/login-json` if you prefer JSON; both exist.

3.4 lib/ws.ts — reconnecting client

```

export function connectMarket(symbols: string[], onTick: (t: Tick) => void) {
  let ws: WebSocket | null = null, closed = false, attempt = 0, timer: any;
  const url = `${process.env.NEXT_PUBLIC_WS_URL}/ws/market?symbols=${symbols.join(",")}`;
  const open = () => {
    ws = new WebSocket(url);
    ws.onopen = () => { attempt = 0; };
    ws.onmessage = (e) => { const m = JSON.parse(e.data); if (m.type === "tick") onTick(m); };
  };
  ws.onclose = () => { if (!closed) timer = setTimeout(open, Math.min(1000 * 2 ** attempt++, 15000)); };
  ws.onerror = () => ws?.close();
  };
  open();
  return () => { closed = true; clearTimeout(timer); ws?.close(); }; // call this in
  useEffect cleanup
}

```

Always return the cleanup and call it — React 18 StrictMode mounts effects twice in dev, and without cleanup you get duplicate sockets and doubled ticks.

3.5 Pages — what each one must show

`/login`, `/register` — form, inline field errors, backend `detail` on failure, redirect to `/` on success.

`/ dashboard` — account summary (cash / held / available / portfolio value), a table of all 8 symbols with live price and 24 h change (green/red, flash on tick), a link into each symbol.

`/market/[symbol]` — the centrepiece:

- Candlestick chart, `1m` / `5m` toggle. Load history from `/api/market/candles`, then apply live ticks with `series.update({time, open, high, low, close})` on the current bucket.
- Order book: bids (green) and asks (red), aggregated per level, with a depth bar. Poll `/api/book/{symbol}` every 2 s (the book has no WS feed — that is fine and documented).

- Recent trades list.
- **Order ticket:** BUY/SELL toggle, LIMIT/MARKET toggle, quantity, price (hidden for MARKET), estimated cost = `price × qty`, available balance shown next to it, and for SELL the available share count. Disable submit while in flight. On success show the order id and refresh the orders list; on 409 show `detail` (e.g. "insufficient buying power") in red.

`/portfolio` — holdings table (symbol, qty, available, avg cost, last price, market value, unrealized P&L, realized P&L) with totals, and colour-coded P&L. Show a "prices may be delayed" note when `price_stale` is true.

`/orders` — open orders (cancellable) and history, with status badges; poll every 3 s while any order is `NEW/PARTIALLY_FILLED/CANCEL_PENDING`. Cancel button → `DELETE`, then optimistic `CANCEL_PENDING` badge.

`/wallet` — balance card, deposit form (quick amounts 1k/10k/100k), transaction history table.

Navbar — logo, nav links, live available balance, notification bell (poll `/api/notifications/unread-count` every 5 s; dropdown lists the latest 10 and marks read on open), username, logout.

3.6 Chart component — the exact shape that works

```
'use client';
import { createChart, ColorType, IChartApi, ISeriesApi } from 'lightweight-charts';

export default function CandleChart({ symbol, interval }: Props) {
  const ref = useRef<HTMLDivElement>(null);
  const chart = useRef<IChartApi>(); const series = useRef<ISeriesApi<'Candlestick'>>();
  useEffect(() => {
    if (!ref.current) return;
    chart.current = createChart(ref.current, {
      layout: { background: { type: ColorType.Solid, color: '#0b0e14' }, textColor: '#d1d4dc' },
      grid: { vertLines: { color: '#1c2030' }, horzLines: { color: '#1c2030' } },
      timeScale: { timeVisible: true, secondsVisible: false },
      autoSize: true,
    });
    series.current = chart.current.addCandlestickSeries({ // v4 API – see §3.1
      upColor: '#26a69a', downColor: '#ef5350',
      wickUpColor: '#26a69a', wickDownColor: '#ef5350', borderVisible: false,
    });
    return () => { chart.current?.remove(); chart.current = undefined; };
  }, []);
  // load history -> series.current.setData(candles)
  // live tick -> series.current.update({ ...currentBucket, close: price, high: max, low:
  min });
  return <div ref={ref} className="h-[420px] w-full" />;
}
```

Import it with `dynamic(() => import('./CandleChart'), { ssr: false })` — it touches `window` and will break SSR otherwise.

3.7 Frontend Dockerfile

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY frontend/package*.json ./
RUN npm ci
COPY frontend/ ./
ARG NEXT_PUBLIC_API_URL=http://localhost:8000
```

```

ARG NEXT_PUBLIC_WS_URL=ws://localhost:8000
ENV NEXT_PUBLIC_API_URL=$NEXT_PUBLIC_API_URL NEXT_PUBLIC_WS_URL=$NEXT_PUBLIC_WS_URL
RUN npm run build

FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/.next ./.next
COPY --from=builder /app/public ./public
COPY --from=builder /app/package*.json ./
COPY --from=builder /app/node_modules ./node_modules
EXPOSE 3000
CMD ["npx", "next", "start", "-p", "3000"]

```

⚠️ **NEXT_PUBLIC_* is inlined at build time, not read at runtime.** They must be **ARGs** in the build stage (Team A passes them via **build.args** in compose). Setting them only as runtime **environment:** does nothing and the app will call **undefined/api/...**

3.8 Edge cases — Frontend

1. **localStorage on the server.** Next.js App Router renders on the server first. Guard every access with **typeof window !== 'undefined'**, and read the token inside **useEffect**, never during render.
2. **Hydration mismatch** from rendering prices/times during SSR. Any component with live data is **'use client'** and starts from a loading skeleton.
3. **401 anywhere** → clear the token, redirect to **/login**. Handled once, in **api()**.
4. **Money arrives as strings.** **Number("195.5000")** for display, and **never** for arithmetic you send back. When posting an order, send the price string exactly as typed and validated.
5. **Tick-size validation client-side** must mirror the backend: positive, ≤ 2 decimals, and quantity a positive integer. Reject before the request so the user gets an instant error — but still handle the 400.
6. **Double-click submit** → disable the button while the request is in flight and send a **client_order_id** (a **crypto.randomUUID()** generated once per ticket) so a retry can never create two orders.
7. **Portfolio lags the fill** (CQRS, ~ 1 s). After a fill notification, wait ~ 1 s then refetch, and show "updating..." instead of a wrong number.
8. **CANCEL_PENDING is not final.** The order can still come back as **FILLED** if it filled first. Render **CANCEL_PENDING** as "Cancelling...", not "Cancelled".
9. **WebSocket in StrictMode** connects twice in dev → always return the cleanup from **useEffect**.
10. **WS dies** (backend restart) → exponential backoff reconnect, and a small "Live / Reconnecting" indicator in the navbar so the demo is honest.
11. **Empty chart** for a symbol with no trades → "No trading activity yet" panel, not a broken canvas.
12. **Chart data must be ascending and unique by time** or lightweight-charts throws. Sort and de-dupe client-side too — cheap insurance.
13. **429 rate limited** → show "Slow down — too many orders, try again in a minute", don't retry automatically.
14. **503 from the gateway** → "Service temporarily unavailable", with a retry button.
15. **Deposit form** must reject 0, negatives, and more than 2 decimals before submitting.
16. **SELL quantity** must be capped at **available_quantity** from **/api/portfolio**, with the reason shown ("10 of 15 reserved by open orders").
17. **Numbers overflowing the layout** — **tabular-nums** and fixed decimals (2 dp for display, even though the API sends 4).
18. **Direct navigation to a protected page** while logged out → **Protected.tsx** wrapper redirects to **/login?next=<path>**.
19. **Symbol in the URL is user input** — uppercase it and 404 gracefully if it isn't one of the 8.
20. **Dark theme only.** One theme, done well, beats two done at 4 a.m.

4. LOCAL DEV

```
# services/market-data-service/docker-compose.dev.yml
services:
  postgres: { image: postgres:16-alpine, environment: {POSTGRES_USER: mse, POSTGRES_PASSWORD:
mse_pw, POSTGRES_DB: market_db}, ports: ["5436:5432"] }
  redis:    { image: redis:7-alpine, ports: ["6379:6379"] }
  rabbitmq: { image: rabbitmq:3.13-management-alpine, ports: ["5672:5672","15672:15672"] }
```

Leave `DATABASE_REPLICA_URL` empty locally — the fallback puts reads on the primary and everything works. Team A wires the real replica.

PowerShell: `$env:PYTHONPATH=".;.././libs"; uvicorn app.main:app --reload --port 8005` Frontend: `cd frontend; npm run dev` with `.env.local` → `NEXT_PUBLIC_API_URL=http://localhost:8000`, `NEXT_PUBLIC_WS_URL=ws://localhost:8000`.

5. TESTING ALONE

`services/market-data-service/selfcheck.py` — plain asserts:

```
floor_bucket(12:03:47, 1) == 12:03:00      floor_bucket(12:03:47, 5) == 12:00:00
floor_bucket(12:05:00, 5) == 12:05:00      floor_bucket(23:59:59, 5) == 23:55:00
first trade of a bucket    -> o=h=l=c=price, volume=qty
second trade higher        -> high rises, OPEN UNCHANGED, close=new, volume sums
second trade lower         -> low falls, open unchanged
same trade_id twice        -> one trade row, volume counted once
gap fill: buckets 1,2,5 with limit 10 -> 5 candles, 3 and 4 flat at candle-2's close, volume
0
candles output: strictly ascending unique epoch-second ints, OHLC are JS numbers
unknown symbol -> 404
```

Feed fake trades — `services/market-data-service/scripts/fake_trades.py`: publish `N` `trade.executed` events with a random walk, 200 ms apart, so charts move and you can watch the WS in the browser. Also perfect for the demo if Team C is late.

6. INDEPENDENCE PLAN — build the frontend before the backend exists

`frontend/mock-api.py` (~120 lines of FastAPI on port 8000) implementing **every** route in §1.4 with in-memory dicts:

- register/login return a real JWT signed with the same `JWT_SECRET`;
- balance/deposit/portfolio/orders/notifications from dicts;
- candles from a seeded random walk;
- `POST /orders` marks the order `FILLED` after 2 s and mutates the portfolio, so the whole user journey is exercisable;
- `/ws/market` pushes a random-walk tick every second.

Build 100 % of the UI against this. When the real stack comes up at hour 18, change nothing but the port — the contract is identical. **Do this first; do not wait for anyone.**

7. HOUR-BY-HOUR PLAN (24 h)

Hours	Work
0–1	Pull repo, read <code>libs/common</code> , start dev containers.
1–2	<code>frontend/mock-api.py</code> — you are now unblocked forever.
2–7	Market Data Service: models, alembic, read/write split, trade handler, candle upsert, all REST endpoints, gap fill. <code>selfcheck.py</code> green.
7–9	WebSocket + Redis pub/sub fan-out + <code>tick_pump</code> + cache warm-up. Dockerfile. Push <code>feat/team-d-marketdata-frontend</code>.
9–11	Frontend skeleton: Next.js + Tailwind + AuthContext + <code>api.ts</code> + Navbar + login/register.
11–14	Dashboard + <code>/market/[symbol]</code> with the chart working against the mock. The chart is the demo — get it right early.
14–16	Order ticket, order book, recent trades, live ticks.
16–18	<code>/portfolio</code> , <code>/orders</code> , <code>/wallet</code> , notification bell. Push — Team A's merge window.
18–21	Point at the real gateway. Fix contract mismatches. On call for Team A.
21–23	Polish: loading skeletons, error toasts, empty states, responsive layout. <code>SERVICE_NOTES.md</code> . Screenshots: chart, book, portfolio, <code>pg_is_in_recovery()</code> on the read engine.
23–24	Freeze.

8. DEFINITION OF DONE

- ☐ Market Data builds from repo root, `/health` + `/ready` + `/docs` green.
- ☐ Startup logs prove reads go to the replica (`pg_is_in_recovery() = true`) and writes to the primary.
- ☐ `selfcheck.py` passes every case in \$5.
- ☐ A trade event produces: a `trades` row, updated 1m **and** 5m candles, `md:last_price` set, a tick on `md:ticks`, and a message on every open WebSocket — within one second.
- ☐ Replaying the same trade event twice changes no number.
- ☐ `/market/candles` returns ascending unique epoch-second `time` values with numeric OHLC and no gaps.
- ☐ Frontend: register → deposit → chart → place order → see the fill → portfolio updates → notification appears, all without a page reload.
- ☐ Every backend error surfaces its `detail` to the user.
- ☐ `SERVICE_NOTES.md` written, including the documented "candles use numbers, everything else uses strings" deviation.